

LEVEL 3 PROJECT REPORT

GitHub + Jenkins + Nginx CI/CD

Automated Static Website Deployment on AWS EC2

Technology Stack

GitHub • Jenkins • AWS EC2 • Amazon Linux • Nginx • SSH/SCP • GitHub Webhook

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Technology / Service	Role in Project
GitHub	Stores portfolio source files and Jenkinsfile.
Jenkins	Checks out source, validates files and performs deployment.
AWS EC2	Hosts the Jenkins environment and Nginx web server.
Amazon Linux	Provides the Linux environment shown in the EC2 implementation.
Nginx	Serves the deployed HTML/CSS/JavaScript website.
GitHub Webhook	Sends push events from GitHub to Jenkins.
SSH / SCP	Supports Jenkins-to-Nginx server communication and file delivery.
Jenkins Credentials	Stores GitHub and deployment credentials used by Jenkins.

Prepared by Manimaran

Implementation evidence: 28 screenshots + 1 system architecture diagram

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Objectives
- 4. Technologies and Tools
- 5. Prerequisites
- 6. System Architecture
- 7. Complete CI/CD Workflow
- 8. Project Components
- 9. Implementation Procedure
- 10. Testing and Validation
- 11. Troubleshooting and Operational Notes
- 12. Security Considerations
- 13. Advantages
- 14. Limitations
- 15. Future Enhancements
- 16. Key Lessons Learned
- 17. Interview / Viva Explanation
- 18. Conclusion
- Appendix A — Implementation Evidence Inventory

1. Project Overview

This Level 3 project demonstrates a Continuous Integration and Continuous Deployment (CI/CD) workflow for a static portfolio website using GitHub, Jenkins and Nginx on AWS EC2. The source code is maintained in GitHub. A GitHub webhook notifies Jenkins when repository changes are pushed. Jenkins checks out the source, performs the configured validation, and deploys the website to a separate Nginx web server through SSH/SCP. Nginx then serves the website to users over HTTP.

The submitted evidence shows the Jenkins installation and service setup, Jenkins pipeline configuration, GitHub repository and Jenkinsfile, Jenkins credentials, webhook deliveries, successful pipeline executions, SSH authorization configuration, the Nginx web-content directory, and browser-level validation of the deployed AWS DevOps portfolio.

2. Problem Statement

Manual website deployment requires repeatedly obtaining the latest source files, copying them to the web server and checking that the updated site is available. Repeating these activities manually can increase deployment effort and introduce configuration mistakes. This project addresses that workflow by using GitHub as the source repository, Jenkins as the automation server, a webhook as the event trigger, and SSH/SCP-based delivery to the Nginx web server.

3. Objectives

- Maintain the portfolio website source code in GitHub.
- Use Jenkins as the CI/CD automation server.
- Trigger the Jenkins pipeline from GitHub push events using a webhook.
- Check out the latest source code automatically.
- Perform a basic website validation before deployment.
- Transfer the website files securely from Jenkins to the Nginx EC2 server.
- Serve the deployed website using Nginx.
- Provide visible Jenkins stages and console output for pipeline verification.
- Validate the final website through a browser.

4. Technologies and Tools

Technology / Service	Purpose	Role in Project
GitHub	Source control	Stores portfolio source files and Jenkinsfile.
Jenkins	CI/CD automation	Checks out source, validates files and performs deployment.
AWS EC2	Compute	Hosts the Jenkins environment and Nginx web server.
Amazon Linux	Operating system	Provides the Linux environment shown in the EC2 implementation.
Nginx	Web server	Serves the deployed HTML/CSS/JavaScript website.
GitHub Webhook	Event trigger	Sends push events from GitHub to Jenkins.
SSH / SCP	Secure transfer	Supports Jenkins-to-Nginx server communication and file delivery.
Jenkins Credentials	Credential handling	Stores GitHub and deployment credentials used by Jenkins.

5. Prerequisites

- AWS account with access to EC2 and the required security-group configuration.
- A Jenkins EC2 instance with Jenkins installed and running.
- A separate EC2 instance configured as the Nginx web server.
- Git installed and a GitHub repository containing the portfolio website.
- A Jenkinsfile defining the CI/CD stages.
- SSH connectivity from Jenkins to the Nginx EC2 host.
- Network access to Jenkins and the web server on the required ports.
- Appropriate Jenkins credentials for GitHub and SSH-based deployment.

6. System Architecture

The supplied architecture separates the CI/CD controller from the web-serving environment. The developer pushes code to GitHub. GitHub sends a webhook event to Jenkins running on EC2. Jenkins checks out the source and runs the configured Checkout and Test stages. The Deploy stage uses SSH/SCP to deliver the website to the Web EC2 instance. The Web EC2 instance runs Amazon Linux and Nginx, which serves the website to the user.

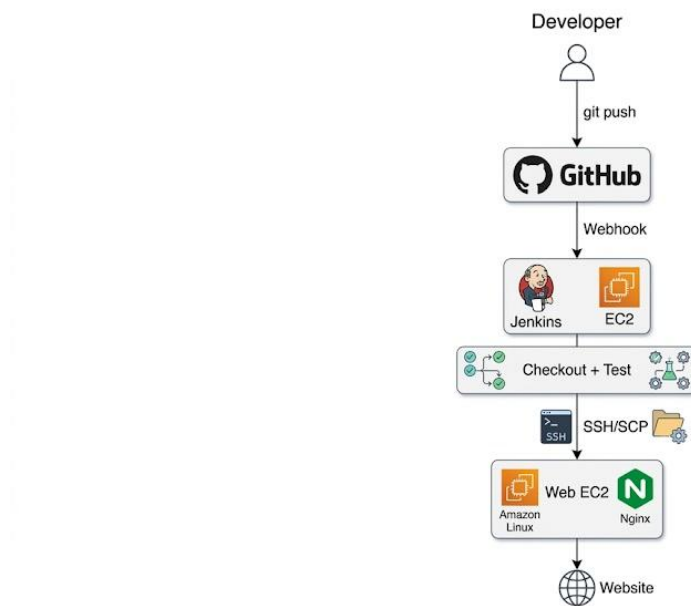


Figure 1. Complete Level 3 GitHub → Jenkins → Nginx CI/CD system architecture.

6.1 Architecture Flow

Step	Flow	Purpose
1	Developer → GitHub	Application changes are committed and pushed to the repository.
2	GitHub → Jenkins	The webhook notifies Jenkins about the push event.
3	Jenkins → Checkout	Jenkins retrieves the latest repository contents.
4	Checkout → Test	The pipeline performs the configured basic website validation.
5	Jenkins → SSH/SCP → Web EC2	The deployment stage transfers the website to the Nginx host.
6	Web EC2 → Nginx	Nginx serves the deployed website files.
7	Nginx → User	The user accesses the portfolio website through the web endpoint.

7. Complete CI/CD Workflow

1. Developer updates the portfolio source code and pushes the change to GitHub.
2. GitHub generates a push webhook event for the configured Jenkins endpoint.
3. Jenkins receives the webhook and starts the GitHub-Nginx-CI-CD pipeline.
4. Jenkins checks out the repository using the configured GitHub credential and master branch.
5. The Test stage performs basic HTML validation, including checking that index.html exists.
6. The Deploy stage uses the configured SSH deployment credential and remote connection.
7. The website files are delivered to the Nginx EC2 web-content environment.
8. Nginx serves index.html together with the CSS, JavaScript and other website assets.
9. Jenkins records the pipeline stage results and console output.
10. Successful browser access provides application-level deployment validation.

8. Project Components

GitHub Repository: Source repository for the AWS DevOps portfolio and CI/CD files.

Jenkinsfile: Defines the declarative pipeline, including Checkout, Test and Deploy stages.

Jenkins Pipeline: The GitHub-Nginx-CI-CD job coordinates the automated workflow.

GitHub Webhook: Connects repository push events to Jenkins.

Jenkins Credentials: Provides GitHub and SSH deployment authentication without exposing secret values in the pipeline.

Nginx EC2 Server: Separate web server that receives the deployment and serves the portfolio.

Nginx Web Directory: The evidence shows the deployed files under `/usr/share/nginx/html/`.

Portfolio Website: The deployed site contains HTML, CSS and JavaScript assets and is validated through a browser.

9. Implementation Procedure

The following 28 steps are presented in the chronological order. Each instruction/action is placed before its corresponding image.

Step 1 – Create the Jenkins EC2 instance

Instruction / Action: The AWS EC2 console shows the Jenkins server instance in the Mumbai region with the instance in the running state.

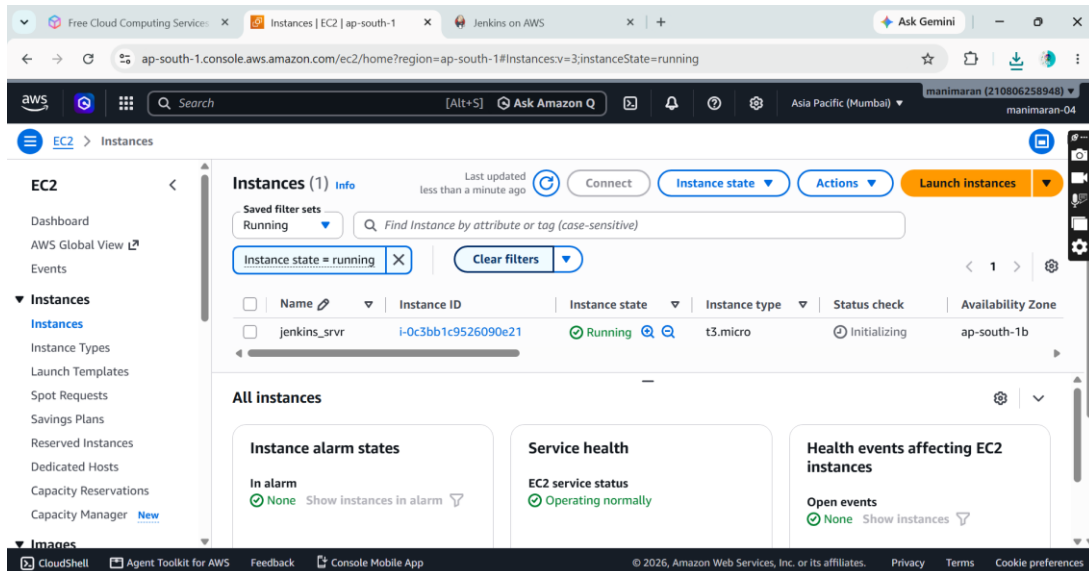


Figure 2. Create the Jenkins EC2 instance.

Step 2 – Verify Jenkins installation and Java environment

Instruction / Action: The EC2 terminal shows Jenkins package installation and the Java runtime version used by the Jenkins environment. A mistyped Java option is followed by the correct version command.

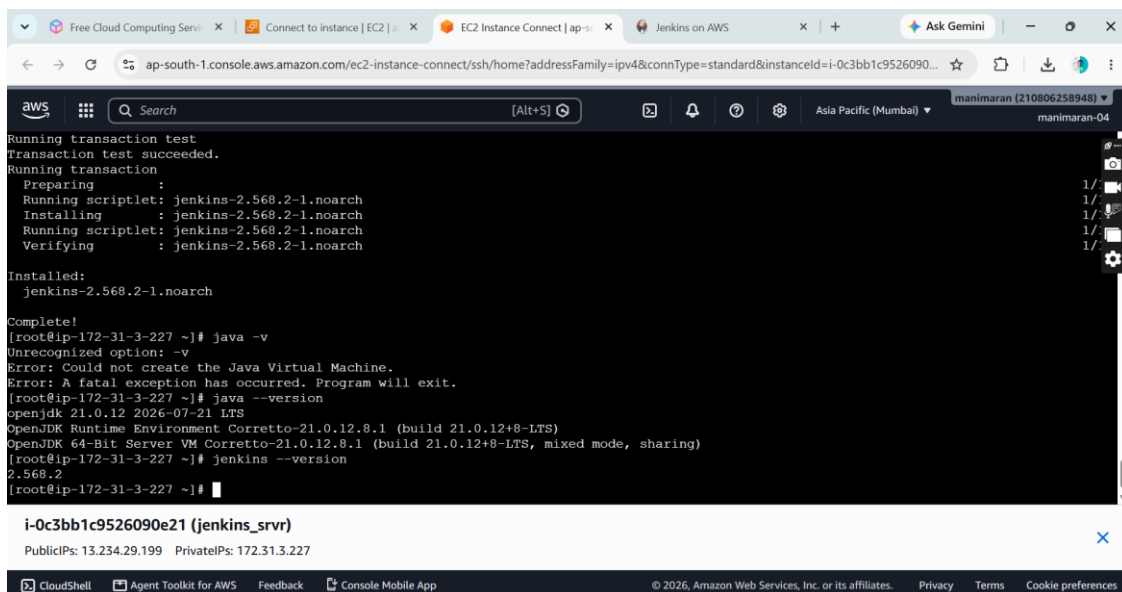


Figure 3. Verify Jenkins installation and Java environment.

Step 3 – Enable and start Jenkins

Instruction / Action: The EC2 terminal shows systemctl enable jenkins and systemctl start jenkins, followed by service-status output confirming Jenkins is active.

```

[root@ip-172-31-3-227 ~]# sudo systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
[root@ip-172-31-3-227 ~]# sudo systemctl start jenkins
[root@ip-172-31-3-227 ~]# sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Wed 2026-08-26 05:21:00 UTC; 9s ago
Main PID: 26637 (java)
Tasks: 43 (limit: 1059)
Memory: 373.2M
CPU: 20.901s
CGroup: /system.slice/jenkins.service
└─26637 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Aug 26 05:20:53 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: [LF]>
Aug 26 05:20:53 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: [LF]>
Aug 26 05:20:53 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: [LF]>
Aug 26 05:20:53 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: [LF]>
Aug 26 05:21:00 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:00.539+0000 [id=31] INFO jenkins
Aug 26 05:21:00 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:00.578+0000 [id=24] INFO hudson.
Aug 26 05:21:00 ip-172-31-3-227.ap-south-1.compute.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Aug 26 05:21:02 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:02.176+0000 [id=50] INFO h.m.Down
Aug 26 05:21:02 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:02.179+0000 [id=50] INFO hudson.
Aug 26 05:21:05 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:05.646+0000 [id=71] WARNING h.n.
  
```

i-0c3bb1c9526090e21 (jenkins_srvr)
PublicIPs: 13.234.29.199 PrivateIPs: 172.31.3.227

Figure 4. Enable and start Jenkins.

Step 4 – Return to the Jenkins unlock stage

Instruction / Action: The Jenkins setup screen is shown again as part of the initial installation and administrator setup sequence.

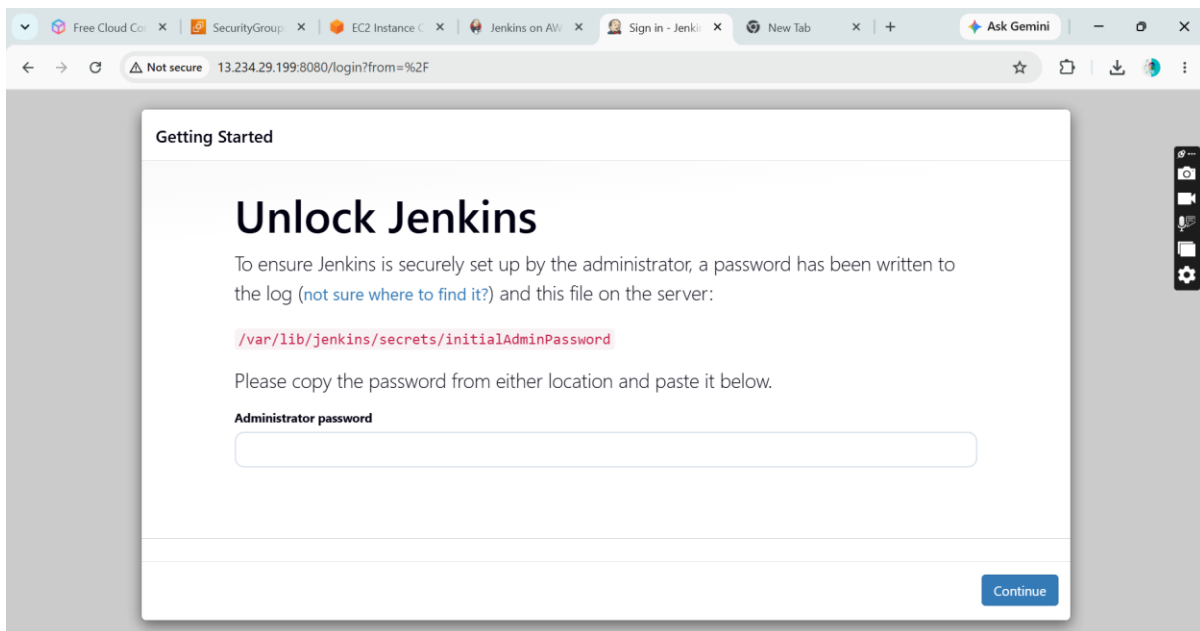


Figure 5. Return to the Jenkins unlock stage.

Step 5 – Copy the Jenkins Initial Admin Password

Instruction / Action: The EC2 terminal shows Jenkins running as a system service, confirming that the Jenkins CI server is active.

The screenshot shows an AWS CloudShell terminal window with the following content:

```

Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Wed 2026-08-26 05:21:00 UTC; 9s ago
Main PID: 26637 (java)
Tasks: 43 (limit: 1059)
Memory: 373.2M
CPU: 20.901s
CGroup: /system.slice/jenkins.service
└─26637 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Aug 26 05:20:53 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: [LF]>
Aug 26 05:20:53 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: [LF]> *****
Aug 26 05:20:53 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: [LF]> *****
Aug 26 05:21:00 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:00.539+0000 [id=31] INFO jenkins
Aug 26 05:21:00 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:00.578+0000 [id=24] INFO hudson
Aug 26 05:21:02 ip-172-31-3-227.ap-south-1.compute.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Aug 26 05:21:02 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:02.176+0000 [id=50] INFO h.m.Down
Aug 26 05:21:02 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:02.179+0000 [id=50] INFO hudson
Aug 26 05:21:05 ip-172-31-3-227.ap-south-1.compute.internal jenkins[26637]: 2026-08-26 05:21:05.646+0000 [id=71] WARNING h.n.
[root@ip-172-31-3-227 ~]# cat /var/lib/jenkins/secrets/initialAdminPassword
cat: failed to parse timestamp or unknown month name: /var/lib/jenkins/secrets/initialAdminPassword
[root@ip-172-31-3-227 ~]# cat /var/lib/jenkins/secrets/initialAdminPassword
ad91d63babf741a298325f7c0a659202
[root@ip-172-31-3-227 ~]#
  
```

Below the terminal output, the instance ID is shown: **i-0c3bb1c9526090e21 (jenkins_svr)**. Public IPs: 13.234.29.199 Private IPs: 172.31.3.227.

Figure 6. Copy the Jenkins Initial Admin Password.

Step 6 – Unlock Jenkins

Instruction / Action: The Jenkins setup wizard requests the administrator password stored on the Jenkins server, using the initialAdminPassword file shown in the workflow.

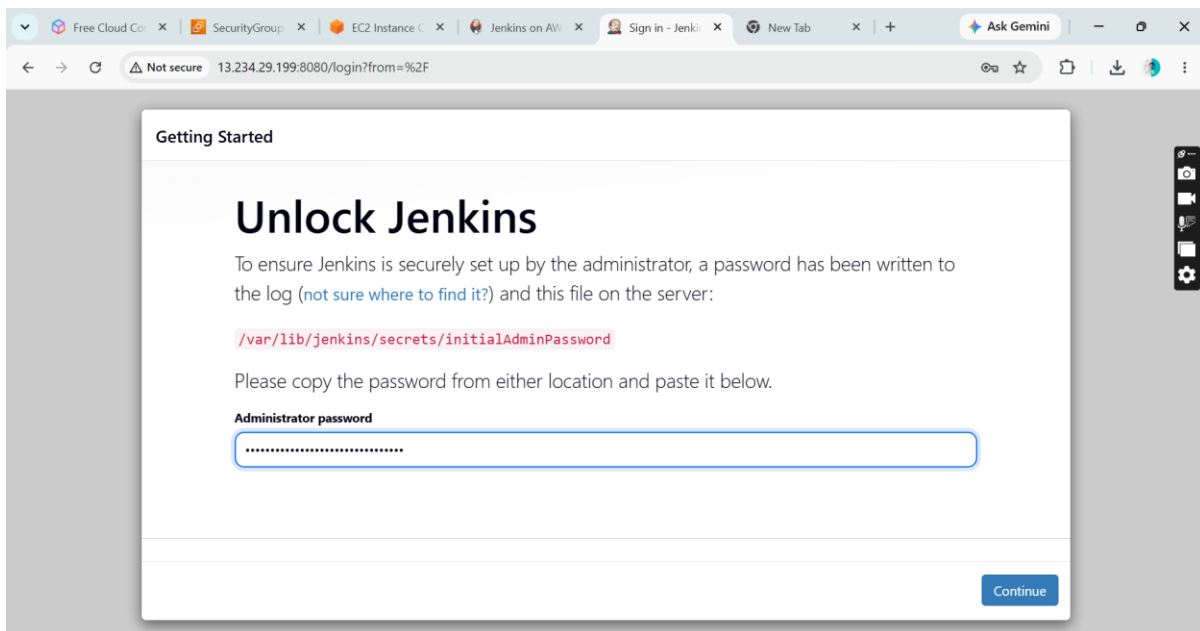


Figure 7. Unlock Jenkins.

Step 7 – Create the first Jenkins administrator

Instruction / Action: The Jenkins setup wizard displays the Create First Admin User form used to establish the initial administrator account.

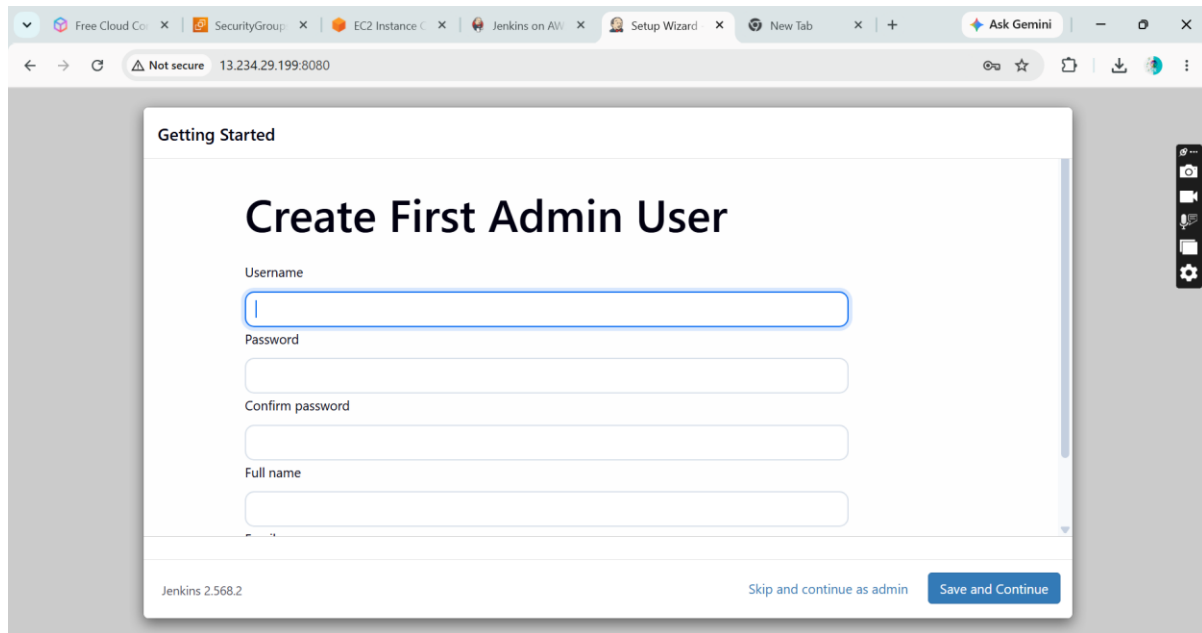
A screenshot of a web browser showing the Jenkins 'Getting Started' page. The main heading is 'Create First Admin User'. Below it are four input fields: 'Username', 'Password', 'Confirm password', and 'Full name'. At the bottom left, it says 'Jenkins 2.568.2'. At the bottom right, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'.

Figure 8. Create the first Jenkins administrator.

Step 8 – Open the Jenkins dashboard after installation

Instruction / Action: The Jenkins dashboard is displayed after initial setup, ready for creation and execution of CI/CD jobs.

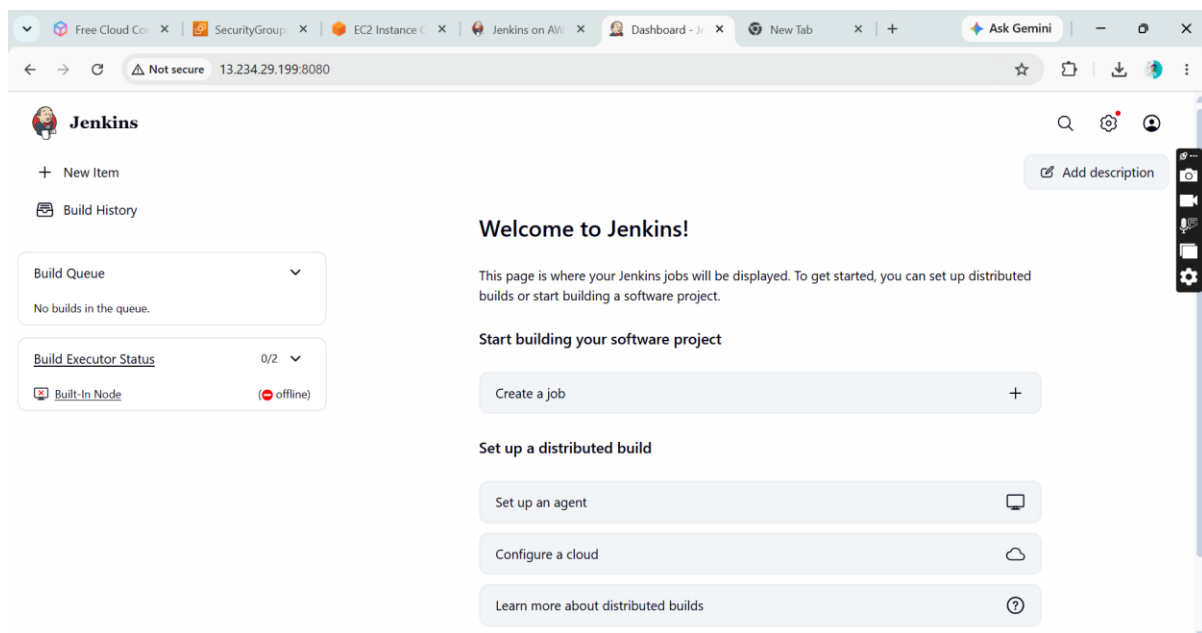


Figure 9. Open the Jenkins dashboard after installation.

Step 9 – Verify the GitHub portfolio repository

Instruction / Action: The GitHub repository contains the portfolio source files and project documentation, including README.md, index.html, script.js and style.css.

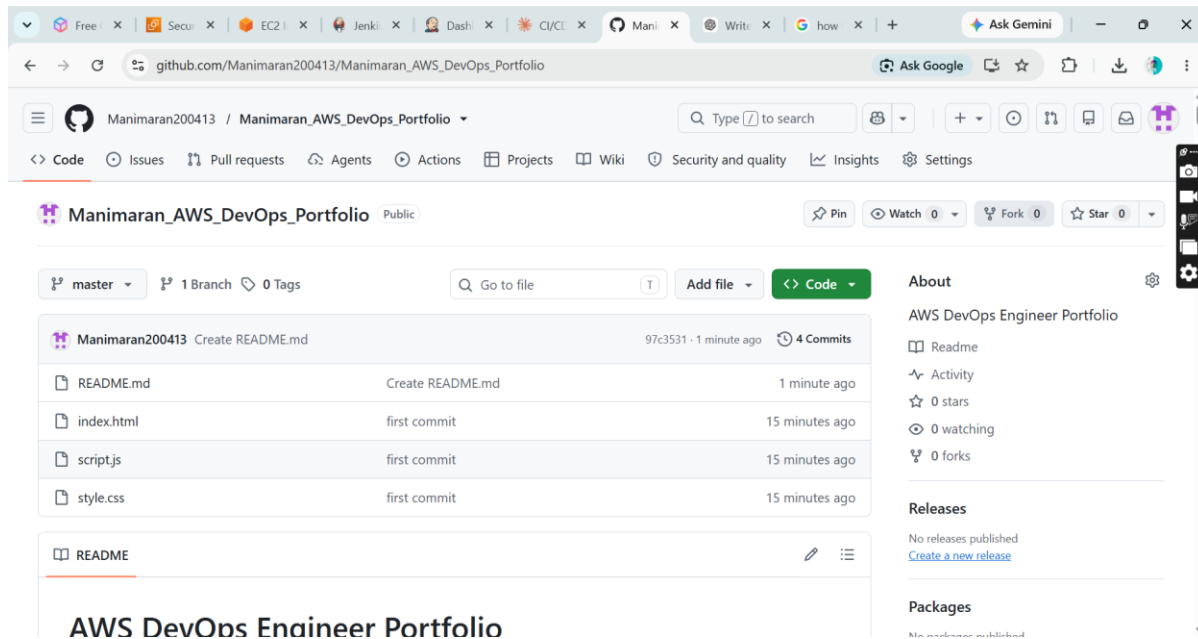


Figure 10. Verify the GitHub portfolio repository.

Step 10 – Verify the Jenkins dashboard and job

Instruction / Action: The Jenkins home page shows the GitHub-Nginx-CI-CD project and build status information.

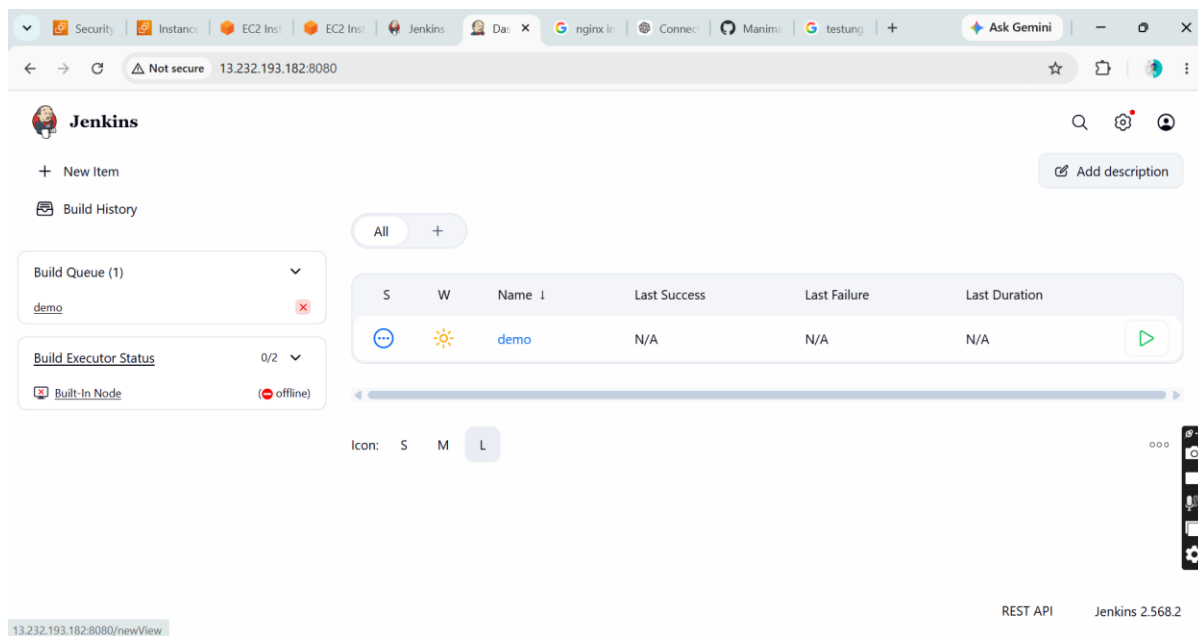


Figure 11. Verify the Jenkins dashboard and job.

Step 11 – Verify the Jenkins built-in node

Instruction / Action: The Jenkins Nodes page shows the built-in node and its controller environment. This confirms the Jenkins execution environment used by the project.

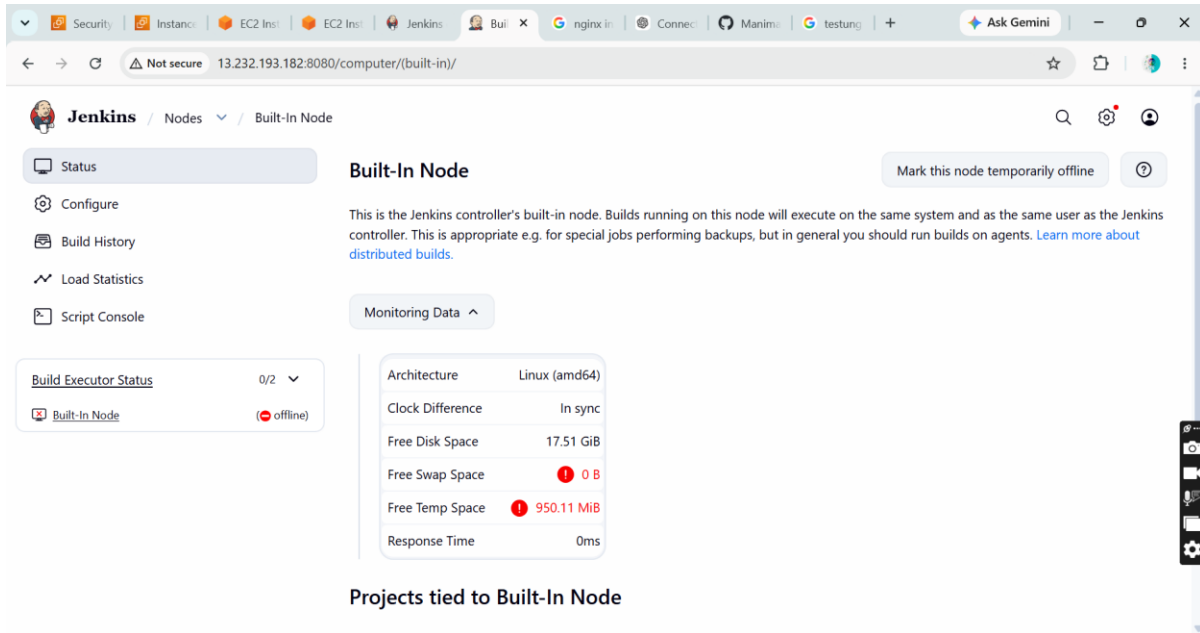


Figure 12. Verify the Jenkins built-in node.

Step 12 – Enable Poll SCM triggering

Instruction / Action: The Jenkins Triggers section shows the poll SCM for checking every minute if there are any new events commit.

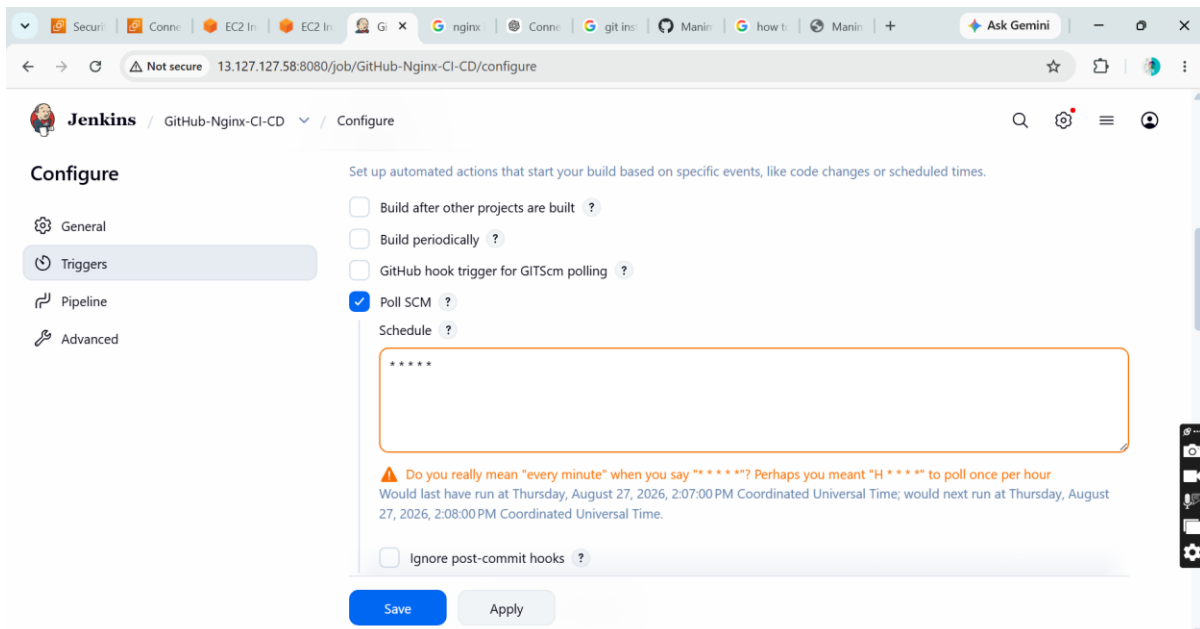


Figure 13. Enable Poll SCM triggering.

Step 13 – Configure Pipeline from SCM

Instruction / Action: The Pipeline section is configured to obtain the Jenkinsfile from source control, using Git and the GitHub repository.

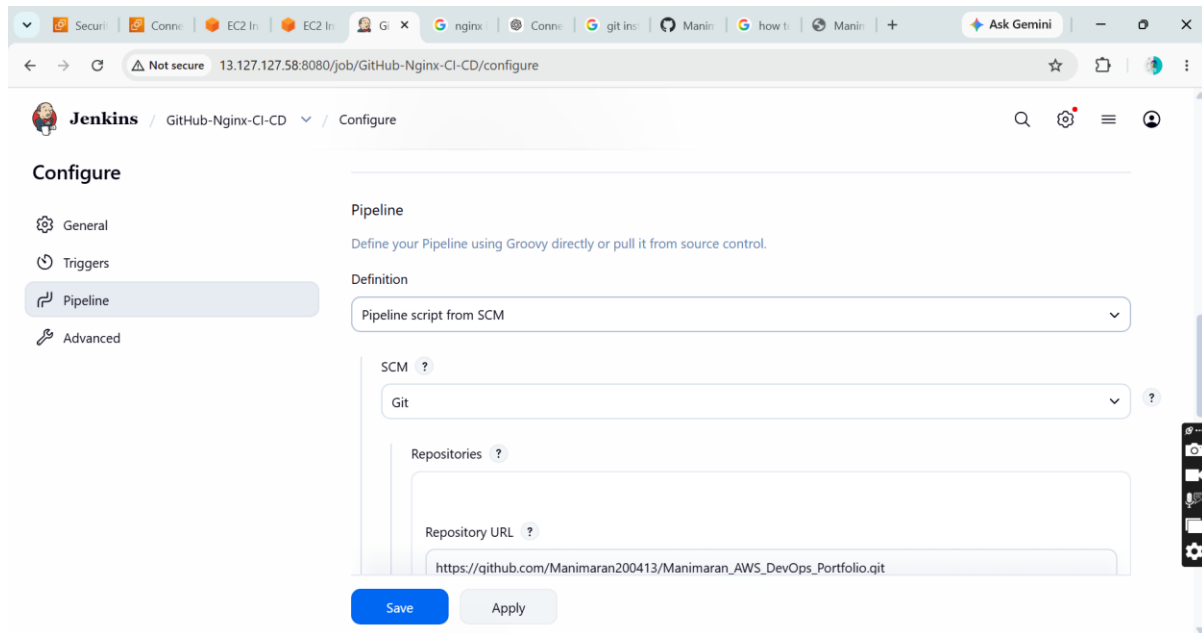


Figure 14. Configure Pipeline from SCM.

Step 14 – Configure GitHub source and branch

Instruction / Action: The Jenkins configuration shows the GitHub repository connection, Git credentials and the master branch specification.

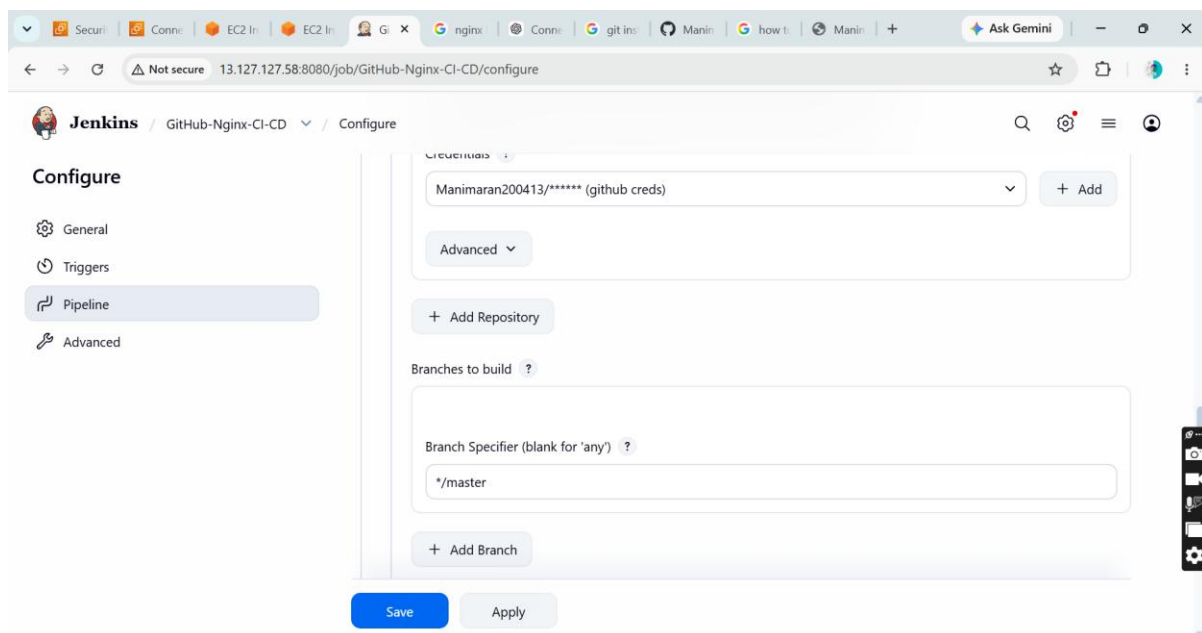


Figure 15. Configure GitHub source and branch.

Step 15 – Configure the Jenkins pipeline script path

Instruction / Action: The Jenkins job configuration shows the Pipeline definition using Jenkinsfile as the script path.

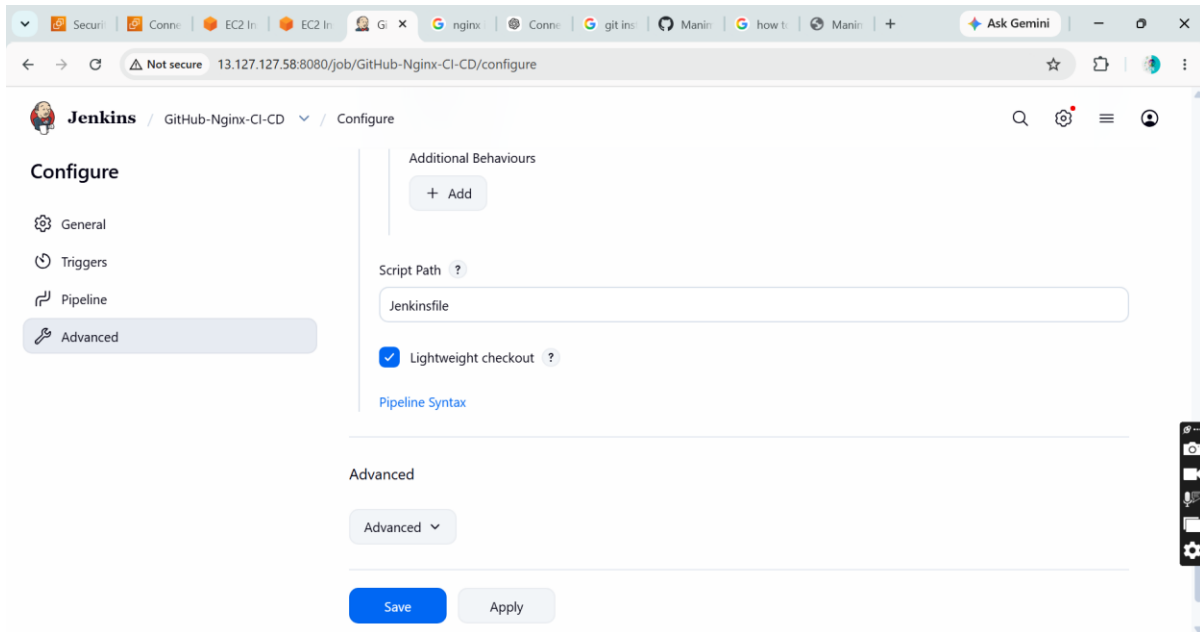


Figure 16. Configure the Jenkins pipeline script path.

Step 16 – Verify Jenkins credentials

Instruction / Action: The Jenkins Credentials page shows the configured GitHub credential and the deployment SSH credential used by the pipeline. Secret values are not reproduced in this report.

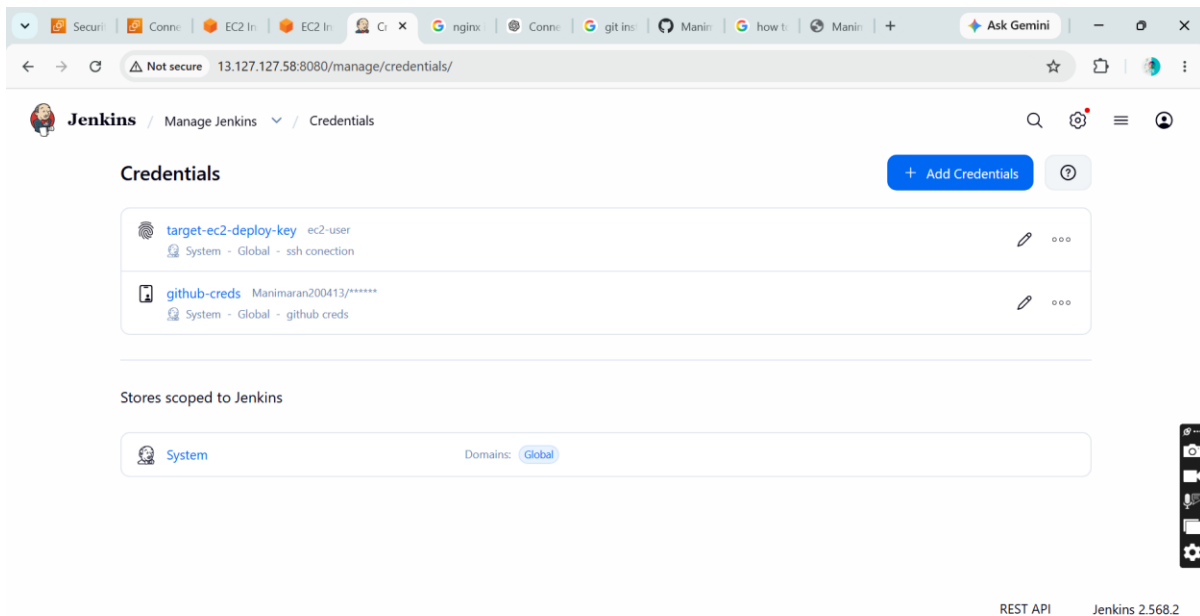


Figure 17. Verify Jenkins credentials.

Step 17 – Verify the Jenkinsfile in GitHub

Instruction / Action: The GitHub repository displays the Jenkinsfile. The visible pipeline definition includes an agent, Checkout and Test stages, the master branch, the repository URL and the github-creds credential.

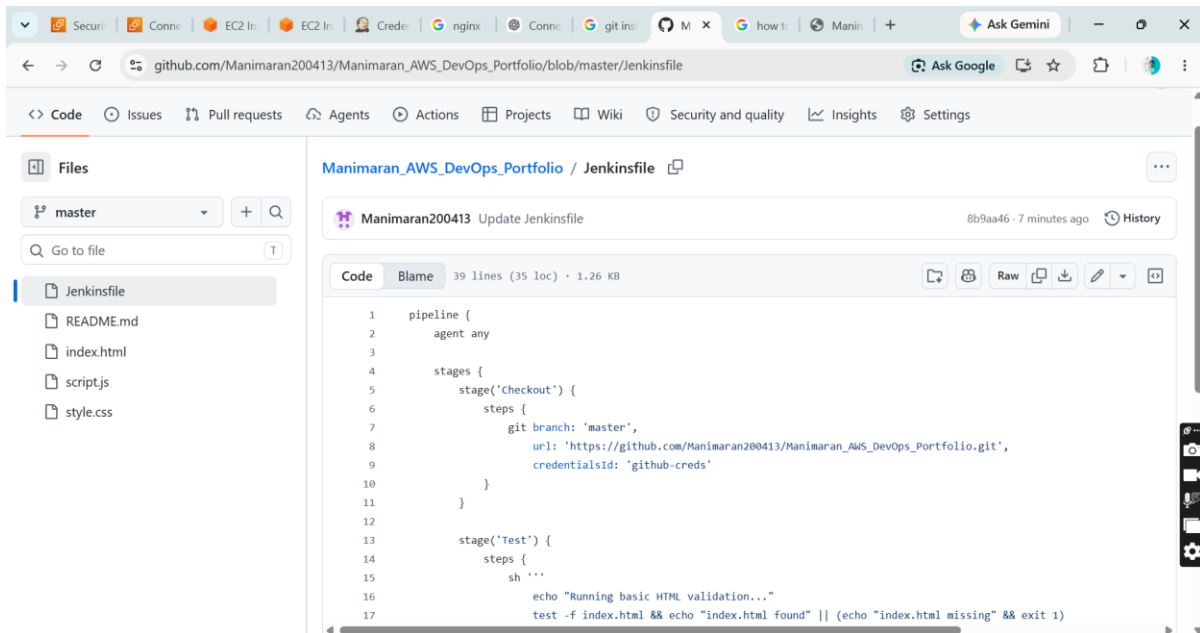


Figure 18. Verify the Jenkinsfile in GitHub.

Step 18 – Verify the Jenkins job from the dashboard

Instruction / Action: The Jenkins dashboard lists the GitHub-Nginx-CI-CD job and its recent build history, confirming that the CI/CD job is configured and has executed.

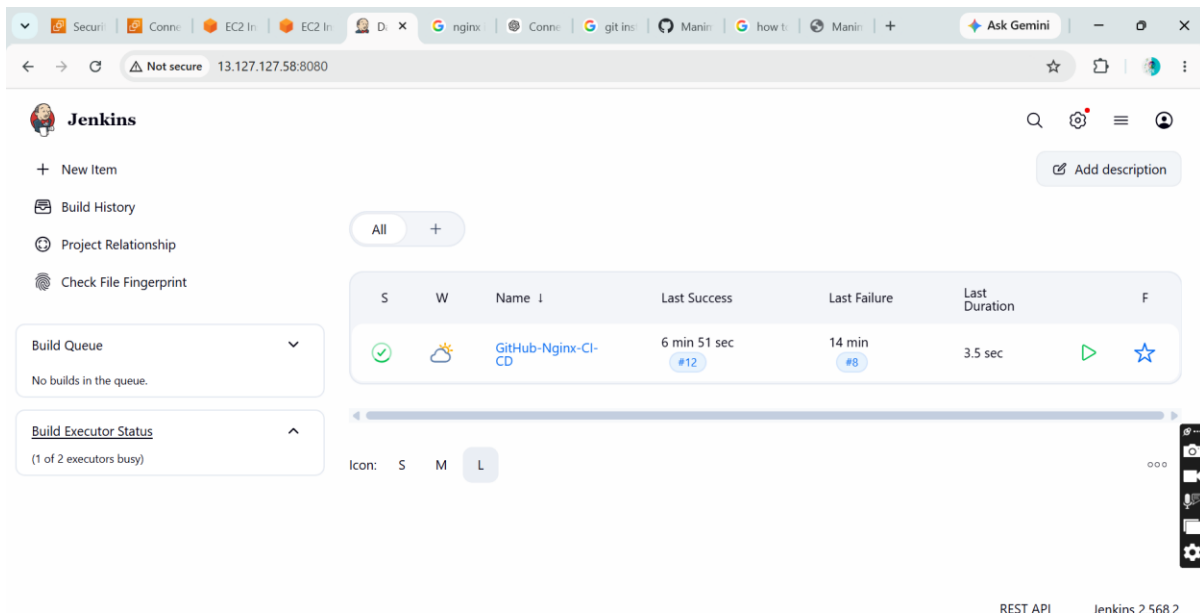


Figure 19. Verify the Jenkins job from the dashboard.

Step 19 – Review the Jenkins Pipeline Stage View

Instruction / Action: The Stage View displays the GitHub-Nginx-CI-CD pipeline with Checkout, Test and Deploy stages and successful stage results.

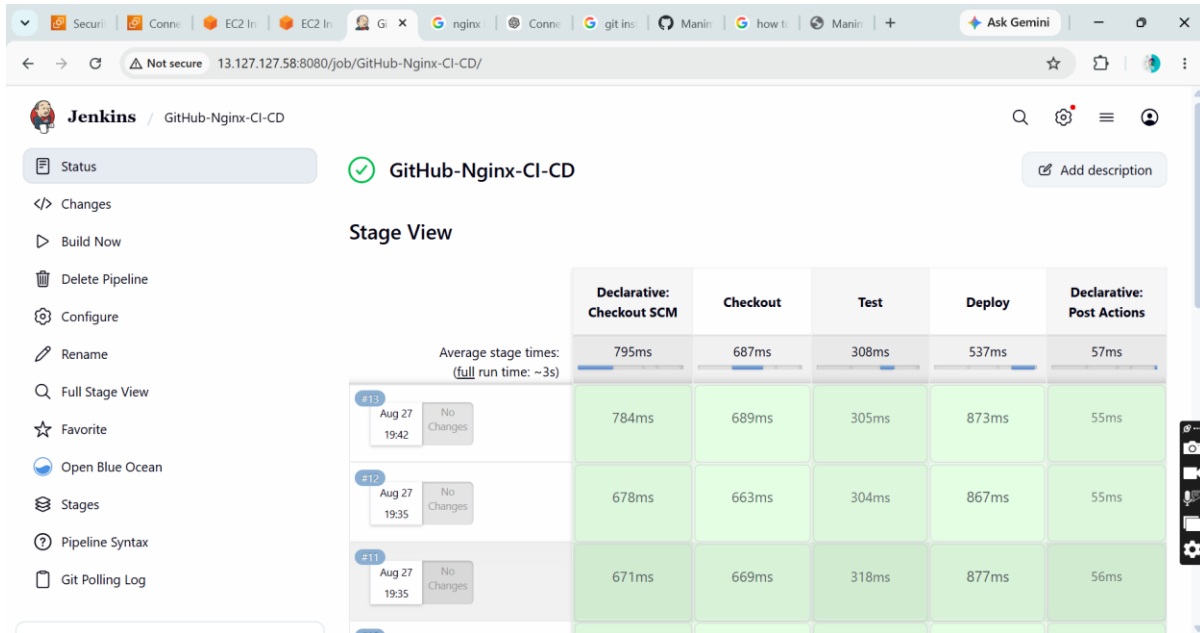


Figure 20. Review the Jenkins Pipeline Stage View.

Step 20 – Verify successful Jenkins deployment

Instruction / Action: The Jenkins console output shows the deployment stage completing and the post-action message “Deployment successful!”, followed by “Finished: SUCCESS”. This provides final pipeline-success evidence.

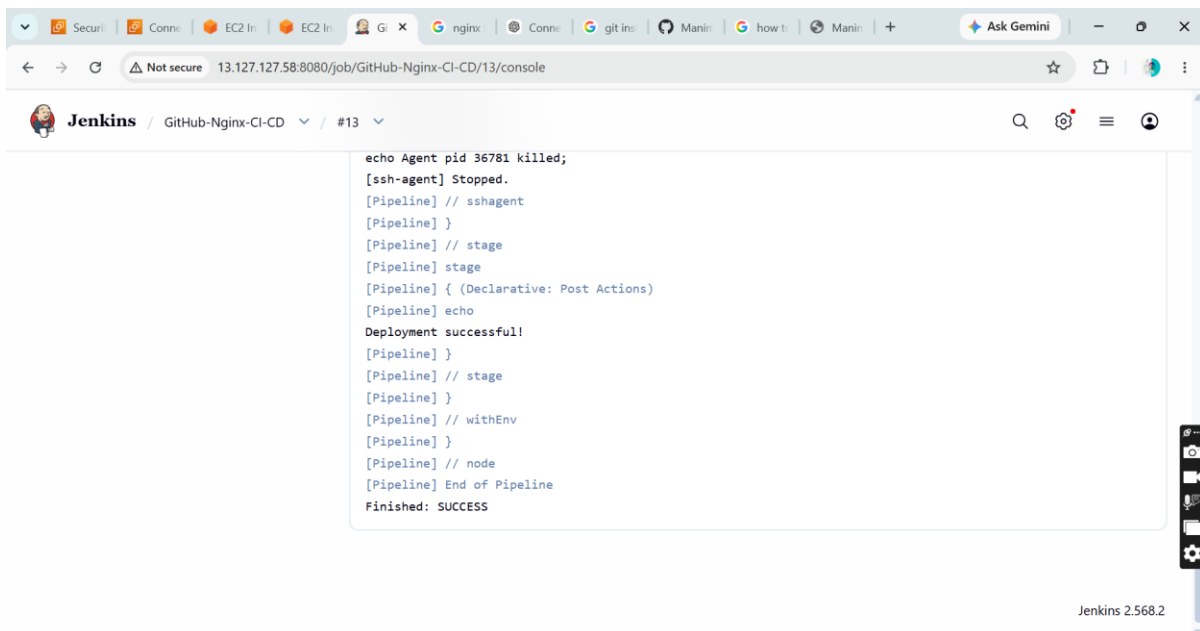


Figure 21. Verify successful Jenkins deployment.

Step 21 – Verify Nginx deployment files and SSH configuration

Instruction / Action: The Nginx EC2 terminal shows SSH authorization-key configuration and the deployed website files under `/usr/share/nginx/html/`, including `Jenkinsfile`, `README.md`, `index.html`, `script.js` and `style.css`.

```

Aug 27 12:47:11 ip-172-31-46-230.ap-south-1.compute.internal sshd-session[28832]: Connection closed by authenticating user ec2-user 172.31.
Aug 27 12:47:47 ip-172-31-46-230.ap-south-1.compute.internal sshd-session[28829]: Connection closed by authenticating user ec2-user 172.31.
[root@ip-172-31-46-230 ~]# ls -l /root/.ssh/jenkins-deploy-key*
ls: cannot access '/root/.ssh/jenkins-deploy-key*': No such file or directory
[root@ip-172-31-46-230 ~]# ls -l ~/.ssh/jenkins-deploy-key*
ls: cannot access '/root/.ssh/jenkins-deploy-key*': No such file or directory
[root@ip-172-31-46-230 ~]# cat /home/ec2-user/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDgIfalgsarTk78udi+nLiDvukziePrEHZ9Gn0eOtzzy2POzrtz7LVBke5TW/14iiu7A/d7S2ZYdbfw7rXLN7WP0zD2wCYVwKYqNDAAv
hwRRSICWsz2ERAZ+s0zDmIuiOm0abWx0/NG5UVTdJMK1121B1d59oTN+dobBdybGx8QImB67hNj61HJNZFSaNVThwsKnYCsM6FFUbnHv+NM3pJ7mTN/17Lj0qL2Z3uvqKvUh2Sf20Sa
vsAxVoNq9noQ55KRh2RqPNkUyVtwpeWatB4vbuaYXBNrjalJlsmfh8KGW/6JrdKP5lQUCiB32nX5M0yYK63C3JEUmj2NF/FFbZJ mumbai
[root@ip-172-31-46-230 ~]# nano /home/ec2-user/.ssh/authorized_keys
[root@ip-172-31-46-230 ~]# chmod 700 /home/ec2-user/.ssh
[root@ip-172-31-46-230 ~]# chmod 600 /home/ec2-user/.ssh/authorized_keys
[root@ip-172-31-46-230 ~]# sudo chown -R ec2-user:ec2-user /home/ec2-user/.ssh
[root@ip-172-31-46-230 ~]# ls -la /usr/share/nginx/html
total 40
drwxr-xr-x. 2 root root 94 Aug 27 14:12 .
drwxr-xr-x. 4 root root 33 Aug 27 12:02 ..
-rw-r--r--. 1 root root 1286 Aug 27 14:12 Jenkinsfile
-rw-r--r--. 1 root root 10122 Aug 27 14:12 README.md
-rw-r--r--. 1 root root 10290 Aug 27 14:12 index.html
-rw-r--r--. 1 root root 852 Aug 27 14:12 script.js
-rw-r--r--. 1 root root 6679 Aug 27 14:12 style.css
[root@ip-172-31-46-230 ~]#
  
```

i-024c9e49f12de307c (nginx-sr)
PublicIPs: 13.206.198.121 PrivateIPs: 172.31.46.230

Figure 22. Verify Nginx deployment files and SSH configuration.

Step 22 – Final browser validation

Instruction / Action: The deployed portfolio is displayed with navigation, profile content, project links and contact information, confirming that the website is available to users.

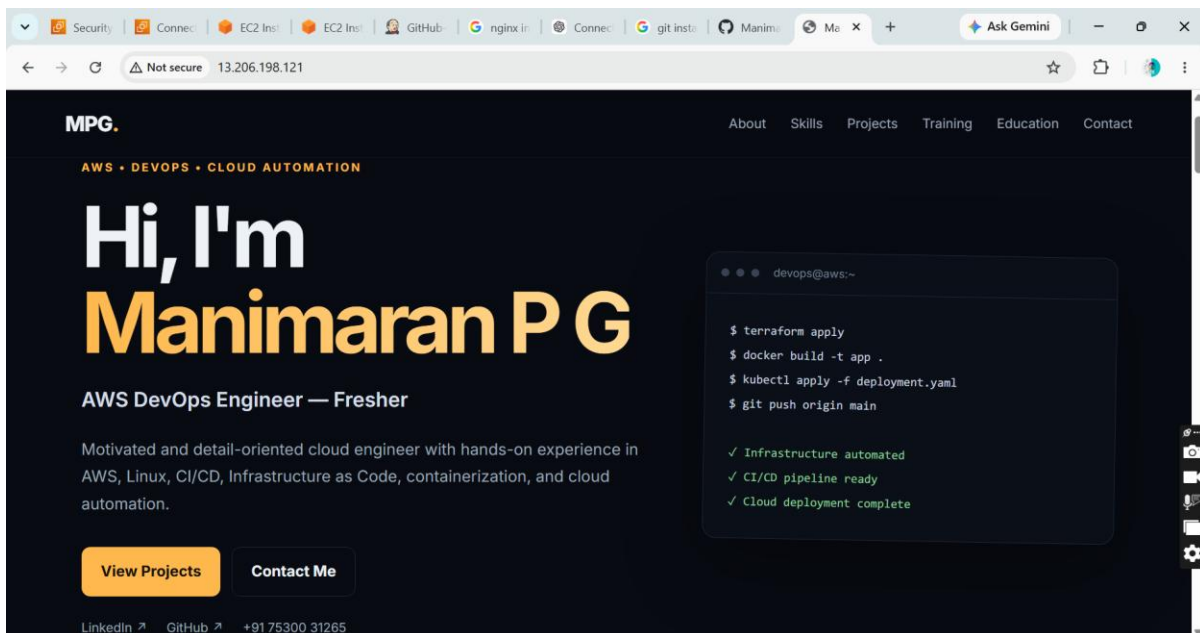


Figure 23. Final browser validation.

Step 23 – Review the portfolio source change

Instruction / Action: The GitHub editor shows index.html being updated in the master branch, including the portfolio title and page structure. This represents a source-code change that can trigger CI/CD.

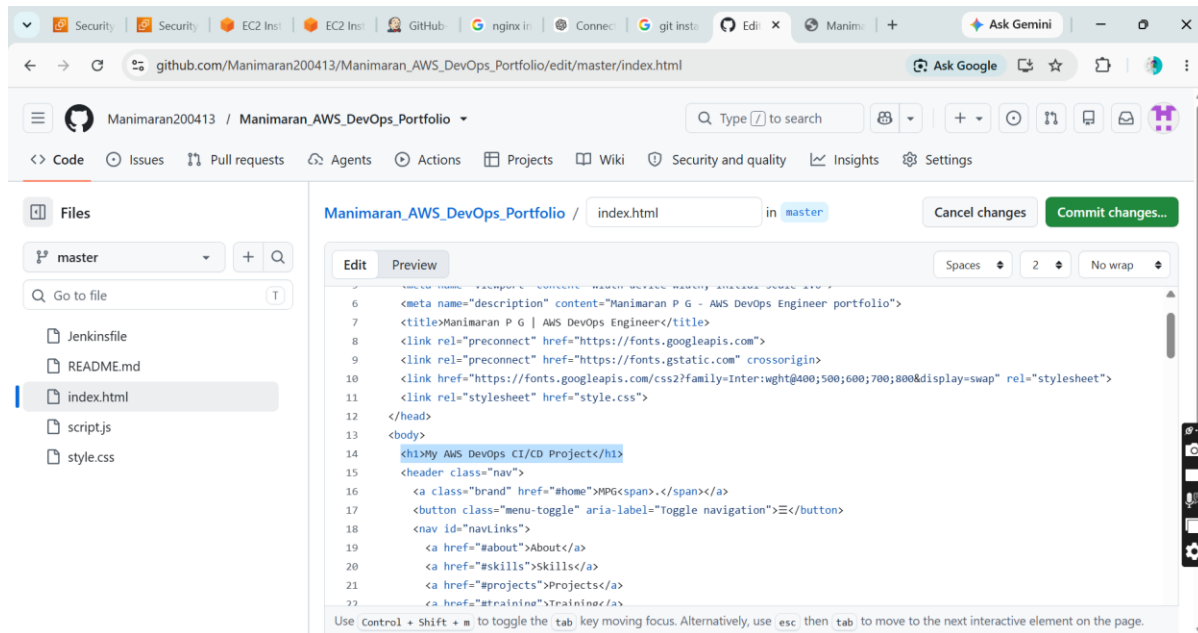


Figure 24. Review the portfolio source change.

Step 24 – Verify the completed CI/CD pipeline

Instruction / Action: The Jenkins Stage View shows successful Checkout, Test and Deploy stages for the GitHub-Nginx-CI-CD job.

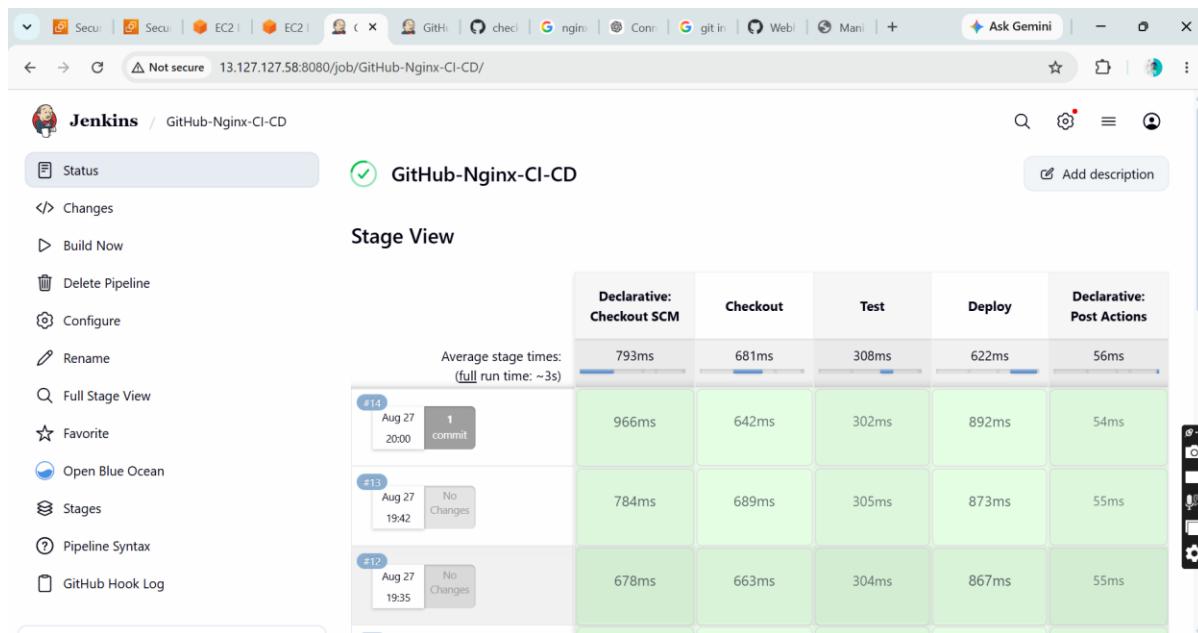


Figure 25. Verify the completed CI/CD pipeline.

Step 25 – Create GitHub webhook delivery

Instruction / Action: The GitHub webhook delivery history shows another push event and a successful HTTP 200 response from Jenkins.

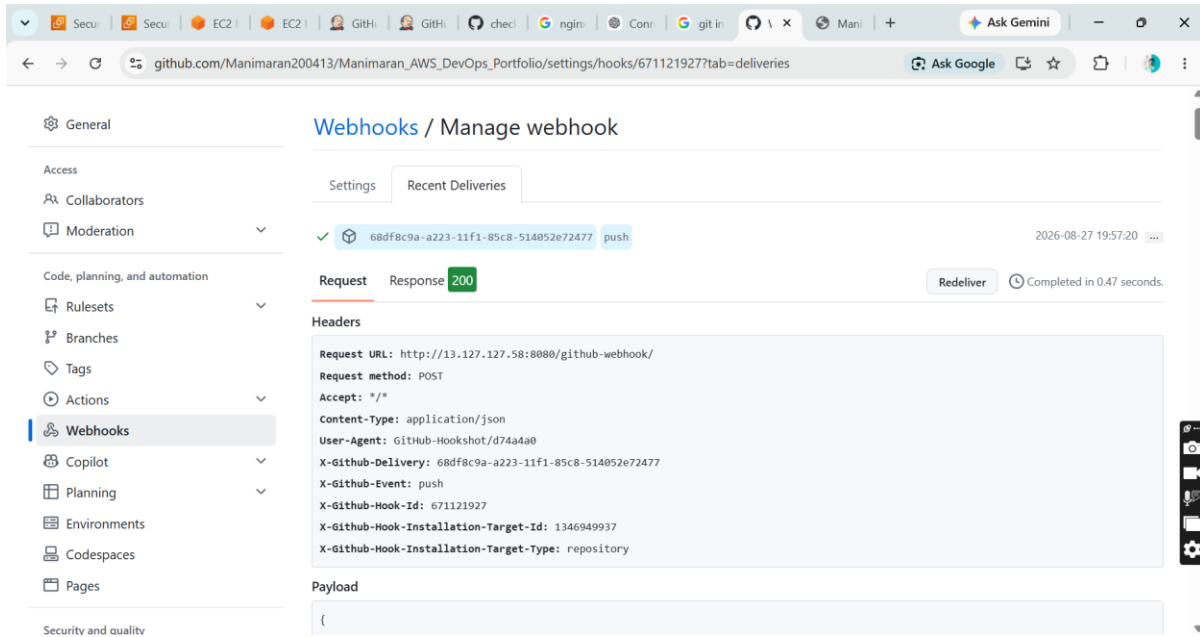


Figure 26. create GitHub webhook delivery.

Step 26 – Validate the updated portfolio page

Instruction / Action: The browser displays the updated portfolio page, showing the deployed website content after source changes.

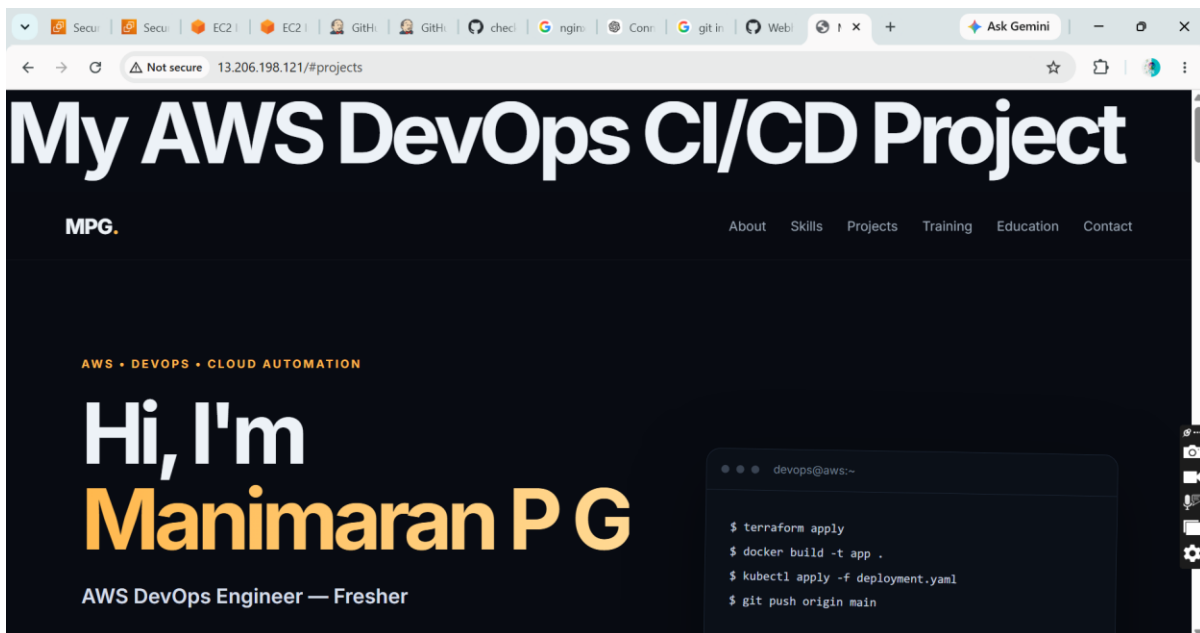


Figure 27. Evidence screenshot – Validate the updated portfolio page.

Step 27 – Verify Another GitHub webhook delivery

Instruction / Action: GitHub Webhooks → Recent Deliveries shows a push event delivered to the Jenkins webhook endpoint with an HTTP 200 response.

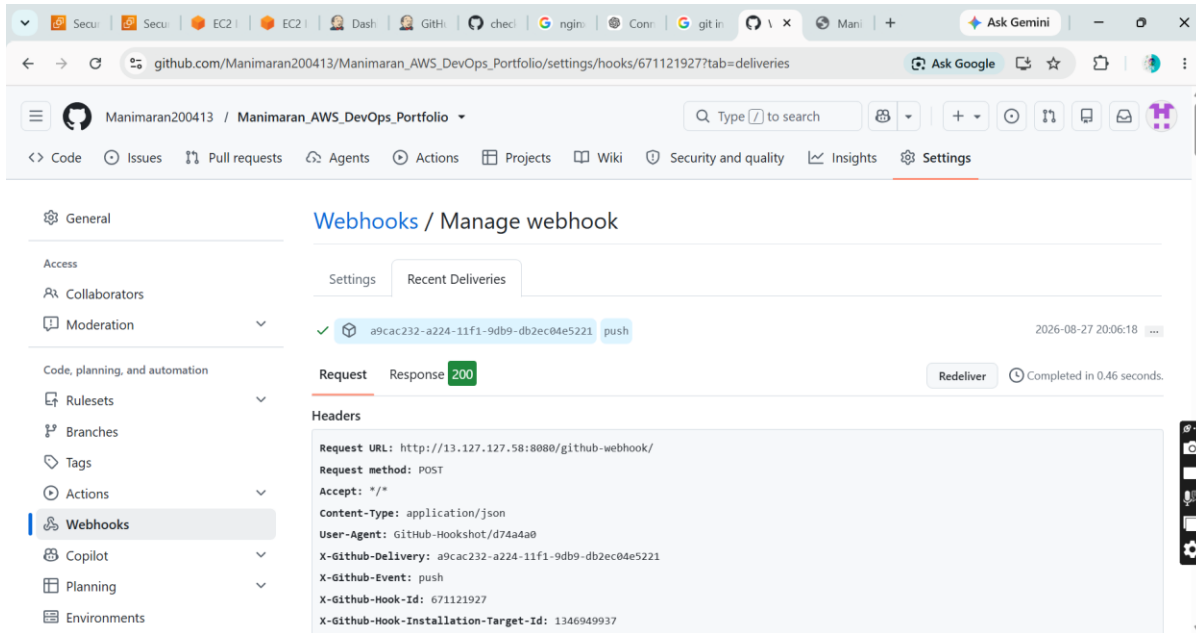


Figure 28. Verify Another GitHub webhook delivery.

Step 28 – Validate the deployed portfolio website

Instruction / Action: The browser displays the AWS DevOps portfolio website after deployment, providing application-level validation of the Nginx-hosted content.

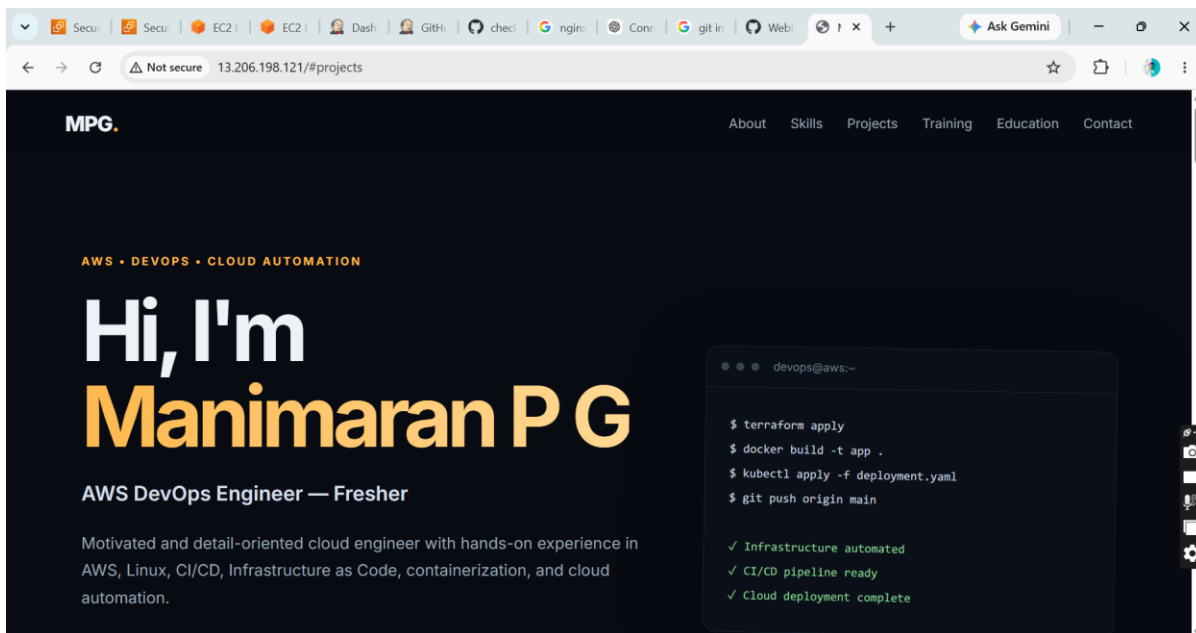


Figure 29. Validate the deployed portfolio website.

10. Testing and Validation

Validation Area	Observed Evidence	Result
GitHub repository	Portfolio source files and Jenkinsfile are visible in the repository.	Passed
Jenkins service	Jenkins service is shown running on the EC2 environment.	Passed
Jenkins pipeline configuration	GitHub repository, master branch, credentials and Jenkinsfile configuration are visible.	Passed
Webhook delivery	Recent GitHub deliveries show push events with HTTP 200 responses.	Passed
Pipeline execution	Checkout, Test and Deploy stages are shown as successful.	Passed
Deployment output	Jenkins console reports “Deployment successful!” and “Finished: SUCCESS”.	Passed
Nginx web files	The Nginx host contains index.html, style.css, script.js and related project files in the web directory.	Passed
Browser validation	The AWS DevOps portfolio page is rendered after deployment.	Passed

11. Troubleshooting and Operational Notes

- The implementation includes Jenkins installation/setup evidence, including the initial administrator and unlock stages.
- The Jenkins terminal evidence shows a mistyped Java version option followed by the correct Java version command; this is command-level troubleshooting rather than a project failure.
- The deployment host evidence shows SSH authorization-key configuration and the Nginx web-content directory, supporting the Jenkins-to-web-server deployment path.
- If a webhook-triggered build does not start, verify the GitHub webhook configuration and the Jenkins trigger setting.
- If SSH deployment fails, verify the Jenkins deployment credential, remote user, authorized_keys configuration, file permissions and network access.
- If the website is not updated, verify the Nginx web directory contents and then retest the browser endpoint.

12. Security Considerations

- Do not commit AWS access keys, passwords, private keys or tokens to GitHub.
- Store GitHub and SSH deployment secrets in Jenkins Credentials rather than hard-coding them in the Jenkinsfile.
- Restrict EC2 security-group rules to only the ports required by the implementation.
- Protect the Jenkins administrative interface with appropriate authentication.
- Use a dedicated deployment user with only the permissions required for the web deployment where practical.
- Use HTTPS/TLS and stronger access controls for a production deployment.
- Sensitive credential and key values are not reproduced as text in this report.

13. Advantages

- Automates a repeatable static website deployment workflow.
- Reduces repeated manual copying of website files.
- Provides visible Jenkins stages and console logs for traceability.
- Uses GitHub webhooks to connect source changes with the CI/CD pipeline.
- Separates CI/CD orchestration from the Nginx web server.
- Provides both pipeline-level and browser-level validation.

14. Limitations

- The demonstrated project uses a direct EC2/Nginx deployment rather than a container orchestration platform.
- The implementation does not demonstrate load balancing or auto scaling for the web tier.
- The evidence represents a learning/project environment rather than a complete production release-management system.
- The supplied workflow uses SSH/SCP-based deployment and therefore requires secure key and permission management.

15. Future Enhancements

- Add Docker containerization and a container registry.
- Introduce an Application Load Balancer and Auto Scaling.
- Add HTTPS/TLS with a managed certificate.
- Add CloudWatch monitoring, logging and alerting.
- Introduce automated security and quality scanning.
- Add separate development, staging and production environments.
- Add approval gates, rollback procedures and deployment health checks.
- Manage the AWS infrastructure using Terraform.

16. Key Lessons Learned

- GitHub webhooks can connect source-code events to Jenkins automation.
- Jenkins declarative pipelines provide a repeatable structure for checkout, validation and deployment.
- SSH/SCP deployment requires correct credentials, authorized keys, permissions and network connectivity.
- A CI/CD implementation should be validated at both the pipeline level and the application level.
- Keeping credentials outside source code is an essential DevOps security practice.

17. Interview / Viva Explanation

“I implemented a Level 3 CI/CD pipeline for a static AWS DevOps portfolio website using GitHub, Jenkins and Nginx on AWS EC2. The developer pushes the website source to GitHub, and a GitHub webhook triggers Jenkins. Jenkins checks out the master branch, performs basic HTML validation and then deploys the website to a separate Nginx EC2 server using SSH/SCP. Nginx serves the deployed HTML, CSS and JavaScript files. I verified the implementation using GitHub webhook delivery results, Jenkins Stage View and console output, the Nginx web-directory contents and browser-level validation of the portfolio.”

18. Conclusion

The Level 3 project demonstrates a practical GitHub-to-Jenkins-to-Nginx CI/CD workflow on AWS EC2. GitHub provides source control, the webhook provides event-driven triggering, Jenkins coordinates checkout and validation, SSH/SCP provides deployment communication, and Nginx serves the resulting website. The supplied evidence documents the installation and configuration of Jenkins, pipeline setup, GitHub integration, webhook deliveries, successful pipeline execution, deployment-host configuration and final browser validation.

The demonstrated delivery path is:

Developer → GitHub → Webhook → Jenkins → Checkout → Test → SSH/SCP → Nginx EC2 → Nginx → Portfolio Website → User.

Appendix A — Implementation Evidence Inventory

Figures	Count	Description
Figure 2 - 21	20 screenshots	Initial Jenkins installation, configuration, GitHub repository and environment evidence.
Figure 22 - 29	8 screenshots	Webhook, pipeline, source-change, browser validation and final Nginx deployment evidence.
Figure 1	System architecture diagram	Supplied Level 3 GitHub → Jenkins → Nginx architecture.

End of Level 3 Project Documentation